Apache CXF - Creating JAX-WS Server

We're creating a maven project using Eclipse. Go to **New > Maven Project**. Check the **Create a simple project** option. and choose the following properties –

- **groupid** – com.tutorialspoint
- **artifactId** – cxf-pojo
- **version** – 1.0

After the completion of the wizard, you will find the appropriate folder structure created in your current folder along with pom.xml file.

You will add the CXF dependencies in the **pom.xml**. For your ready reference, we have given below the pom.xml file for the project that we created on our machine.

```
<?xml version = "1.0" encoding = "UTF-8"?>
<project xmlns = "http://maven.apache.org/POM/4.0.0" xmlns:xsi =
"http://www.w3.org/2001/XMLSchema-instance" xsi:schemaLocation =
"http://maven.apache.org/POM/4.0.0 http://maven.apache.org/xsd/maven-
4.0.0.xsd">
   <modelVersion>4.0.0</modelVersion>
   <groupId>com.tutorialspoint</groupId>
   <artifactId>cxf-pojo</artifactId>
   <version>1.0</version>
   <packaging>jar</packaging>
   <properties>
      <project.build.sourceEncoding>UTF-8</project.build.sourceEncoding>
      <maven.compiler.source>24</maven.compiler.source>
      <maven.compiler.target>24</maven.compiler.target>
      <cxf.rt.version>4.1.3</cxf.rt.version>
   </properties>
   <profiles>
```

```
        <profile>
           <id>server</id>
           <build>
              <defaultGoal>test</defaultGoal>
              <plugins>
                 <plugin>
                    <groupId>org.codehaus.mojo</groupId>
                    <artifactId>exec-maven-plugin</artifactId>
                    <version>1.6.0</version>
                    <executions>
                       <execution>
                          <phase>test</phase>
                          <goals>
                             <goal>java</goal>
                          </goals>
                          <configuration>
                             <mainClass>

   com.tutorialspoint.cxf.jaxws.helloworld.Server
                             </mainClass>
                          </configuration>
                       </execution>
                    </executions>
                 </plugin>
              </plugins>
           </build>
        </profile>
     </profiles>
     <dependencies>
   <dependency>
       <groupId>org.apache.cxf</groupId>
       <artifactId>cxf-rt-features-logging</artifactId>
       <version>${cxf.rt.version}</version>
       <scope>test</scope>
   </dependency>
        <dependency>
           <groupId>org.apache.cxf</groupId>
           <artifactId>cxf-rt-frontend-simple</artifactId>
           <version>${cxf.rt.version}</version>
        </dependency>
```

```
        <dependency>
           <groupId>org.apache.cxf</groupId>
           <artifactId>cxf-rt-transports-http</artifactId>
           <version>${cxf.rt.version}</version>
        </dependency>
        <dependency>
           <groupId>org.apache.cxf</groupId>
           <artifactId>cxf-rt-frontend-jaxws</artifactId>
           <version>${cxf.rt.version}</version>
        </dependency>
        <!-- Jetty is needed if you're using the CXFServlet -->
        <dependency>
           <groupId>org.apache.cxf</groupId>
           <artifactId>cxf-rt-transports-http-jetty</artifactId>
           <version>${cxf.rt.version}</version>
        </dependency>
     </dependencies>
  </project>
```

HelloWorld.java

```
package com.tutorialspoint.cxf.jaxws.helloworld;

import jakarta.jws.WebService;

@WebService
public interface HelloWorld {
   String greetings(String text);
}
```

HelloWorldImpl.java

```
package com.tutorialspoint.cxf.jaxws.helloworld;

public class HelloWorldImpl implements HelloWorld {
   @Override
```

```
   public String greetings(String name) {
      return ("hi " + name);
   }
}
```

Server.java

```
package com.tutorialspoint.cxf.jaxws.helloworld;

import org.apache.cxf.feature.LoggingFeature;

import jakarta.xml.ws.Endpoint;

public class Server {
   public static void main(String[] args) throws Exception {
      HelloWorld implementor = new HelloWorldImpl();
      Endpoint.publish("http://localhost:9090/HelloServerPort",
      implementor,
      new LoggingFeature());
      System.out.println("Server ready...");
      Thread.sleep(5 * 60 * 1000);
      System.out.println("Server exiting ...");
      System.exit(0);
   }
}
```

WEB-INF/web.xml

```
<?xml version = "1.0" encoding = "UTF-8"??>
<web-app xmlns = "http://java.sun.com/xml/ns/javaee"
xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance" version="2.5"
xsi:schemaLocation = "http://java.sun.com/xml/ns/javaee
http://java.sun.com/xml/ns/javaee/web-app_2_5.xsd">
   <display-name>cxf</display-name>
   <servlet>
      <description>Apache CXF Endpoint</description>
```

```
      <display-name>cxf</display-name>
      <servlet-name>cxf</servlet-name>
      <servlet-class>
         org.apache.cxf.transport.servlet.CXFServlet
      </servlet-class>
      <load-on-startup>
         1
      </load-on-startup>
   </servlet>
   <servlet-mapping>
      <servlet-name>
         cxf
      </servlet-name>
      <url-pattern>
         /services/*
      </url-pattern>
   </servlet-mapping>
   <session-config>
      <session-timeout>60</session-timeout>
   </session-config>
</web-app>
```

WEB-INF/cxf-servlet.xml

```
<beans xmlns="http://www.springframework.org/schema/beans"
    xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
    xmlns:jaxws="http://cxf.apache.org/jaxws"
    xsi:schemaLocation="http://www.springframework.org/schema/beans
    http://www.springframework.org/schema/beans/spring-beans-2.0.xsd
    http://cxf.apache.org/jaxws
    http://cxf.apache.org/schemas/jaxws.xsd">

   <jaxws:endpoint xmlns:helloworld = "https://www.tutorialspoint.com/"
      id = "helloHTTP"
      address = "http://localhost:9090/HelloServerPort"
      serviceName = "helloworld:HelloServiceService"
      endpointName = "helloworld:HelloServicePort">
   </jaxws:endpoint>
```

```
</beans>
```

Running the HelloWorld Service

Now, you are ready to run the web app. In the command window, run the build script using the following command.

```
mvn clean install
```

```
mvn -Pserver
```

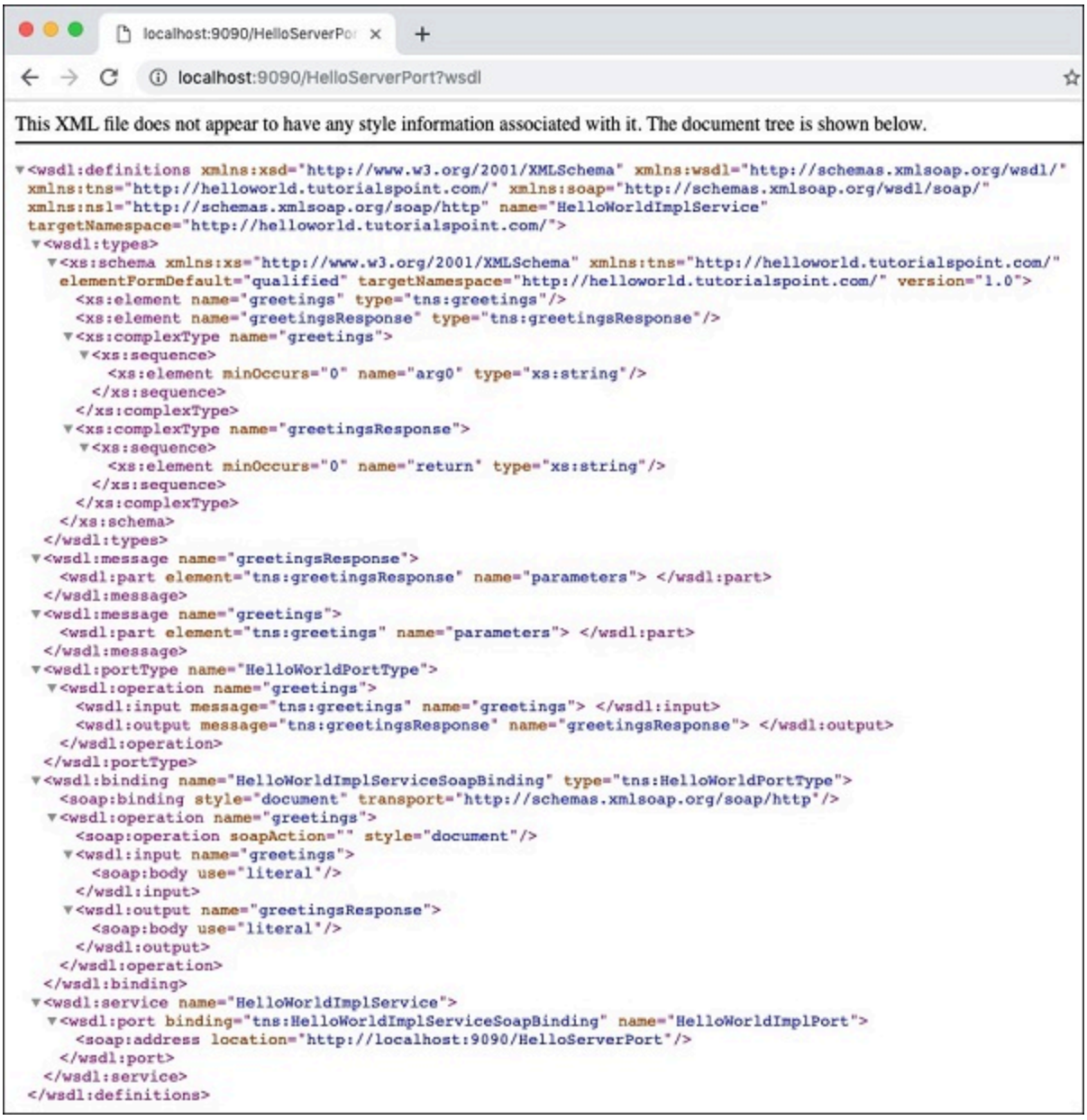
You will see the following message on the console –

```
INFO: Setting the server's publish address to be
http://localhost:9090/HelloServerPort
Server ready
```

Like earlier, you can test the server by opening the server URL in your browser.

As we did not specify any operation, only a fault message is returned to the browser by our application.

Now, try adding the **?wsdl** to your URL and you will see the following output –



So our server application is running as expected. You may use the SOAP Client such as **Postman** described earlier to further test your service.